

# CLAIRE

## Technical Codebook & Engineering Reference

Module Reference, Stage Mapping, API Structure, and  
Configuration Guide

---

**Version:** v5.90.2 (LOCKED)

**Date:** May 3, 2026

**Author:** Craig Morris

**Runtime:** Python 3.11 / 3.13

**Classification:** CONFIDENTIAL

This document is the property of the Claire engineering team. Unauthorized distribution, reproduction, or disclosure is strictly prohibited. This codebook reflects the system architecture as of v5.90.2 and is intended for internal engineering reference only.

---

## Table of Contents

**Section 1:** System Overview

1.1 Technology Stack | 1.2 Project Root Structure | 1.3 Launch Modes | 1.4 Dashboard Logging

## **Section 2:** Module Reference

2.1 ingestion/ | 2.2 connectors/ | 2.3 core/ | 2.4 live\_intelligence/ | 2.5 concepts/ | 2.6 design/ | 2.7 acquirers/ | 2.8 deals/ | 2.9 analysis/ | 2.10 lifecycle/ | 2.11 api/ | 2.12 learning/ | 2.13 mode/ | 2.14 config/ | 2.15 infra/ | 2.16 models/ | 2.17 bridge/

## **Section 3:** Stage-to-Module Mapping

## **Section 4:** API Route Reference

## **Section 5:** Governance Spine Reference

5.1 Lifecycle Runner Execution Flow | 5.2 Stage Contract Structure | 5.3 Threshold Provenance

## **Section 6:** Configuration Reference

6.1 Environment Variables | 6.2 Security Configuration | 6.3 Development vs Production

## **Section 7:** Deployment Reference

7.1 Docker Configuration | 7.2 nginx Configuration | 7.3 CI/CD Pipeline

## **Section 8:** Output Package Reference

8.1 Standard Output Package | 8.2 Enhanced Output Package

## **Section 9:** Run Directory Structure

## **Section 10:** Known Gaps & Future Modules

---

# **Section 1: System Overview**

## **1.1 Technology Stack**

| Component | Technology         | Notes                                                                     |
|-----------|--------------------|---------------------------------------------------------------------------|
| Language  | Python 3.11 / 3.13 | Dual version support confirmed by <code>__pycache__</code> bytecode files |

| Component          | Technology                        | Notes                                                    |
|--------------------|-----------------------------------|----------------------------------------------------------|
| Web Framework      | aiohttp                           | Async HTTP server for high-throughput API operations     |
| Data Validation    | Pydantic 2.13.3                   | Schema enforcement for all data models and API contracts |
| Deployment         | Docker + nginx                    | Containerized deployment with reverse proxy              |
| CI/CD              | GitHub Actions                    | Automated testing, linting, and build verification       |
| Testing            | pytest                            | Unit and integration test suite                          |
| Package Management | pyproject.toml + requirements.txt | PEP 621 compliant with pip fallback                      |
| Version Control    | Git                               | Configured via .gitattributes, .gitignore                |

## 1.2 Project Root Structure

The following table documents every root-level file in the Claire project and its purpose within the system.

| File                | Purpose                                                                                                        |
|---------------------|----------------------------------------------------------------------------------------------------------------|
| main.py             | Application entry point — initializes the runtime, loads configuration, and starts the aiohttp server          |
| .env / .env.example | Environment variables (secrets, API keys, configuration). .env.example provides a template for new deployments |
| pyproject.toml      | Python project configuration and dependency declarations (PEP 621)                                             |
| requirements.txt    | Pip-installable dependency list for environments that do not support pyproject.toml                            |

| File                                        | Purpose                                                                    |
|---------------------------------------------|----------------------------------------------------------------------------|
| <code>pytest.ini</code>                     | Test configuration — markers, test discovery paths, and pytest options     |
| <code>version.json</code>                   | System version tracking — currently v5.90.2                                |
| <code>CHANGELOG.md</code>                   | Version history and change documentation                                   |
| <code>CONTRIBUTING.md</code>                | Contributor guidelines — code style, PR process, branch naming             |
| <code>LICENSE</code>                        | License terms                                                              |
| <code>README.md</code>                      | Project documentation — setup, quickstart, architecture overview           |
| <code>START_CLAIRE.bat</code>               | Standard launch script — default configuration for local development       |
| <code>START_CLAIRE_DESKTOP.bat</code>       | Desktop mode launch — local UI mode                                        |
| <code>START_CLAIRE_LIVE.bat</code>          | Live/production mode launch — full data connections enabled                |
| <code>START_CLAIRE_PORTABLE.bat</code>      | Portable mode launch — self-contained deployment with bundled dependencies |
| <code>claire-syntalion-enhanced.html</code> | Enhanced dashboard interface — browser-based monitoring and control panel  |

## 1.3 Launch Modes

Claire supports four distinct launch modes, each configured via a dedicated batch script. The mode determines data source connectivity, logging verbosity, UI behavior, and deployment constraints.

| Mode            | Script                        | Description                                                                                                                                              |
|-----------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Standard</b> | <code>START_CLAIRE.bat</code> | Default configuration for local development. Uses local data sources, verbose logging, and debug-mode enabled. Suitable for day-to-day engineering work. |

| Mode            | Script                    | Description                                                                                                                                                                                           |
|-----------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Desktop</b>  | START_CLAIRE_DESKTOP.bat  | Desktop application mode with local UI integration. Launches the dashboard interface alongside the backend server for single-machine operation.                                                       |
| <b>Live</b>     | START_CLAIRE_LIVE.bat     | Production/live mode with full data connections. Activates all external connectors (financial, market, news, academic, patent), uses production API keys, structured logging, and strict rate limits. |
| <b>Portable</b> | START_CLAIRE_PORTABLE.bat | Self-contained portable deployment. Bundles all dependencies and configuration for deployment on machines without a pre-configured Python environment.                                                |

## 1.4 Dashboard Logging

Dashboard logs are versioned per release to isolate log output across deployments and facilitate regression analysis. All logs are written to the project root directory.

| Log File                         | Purpose                                    |
|----------------------------------|--------------------------------------------|
| dashboard_8765.out.log           | Base dashboard standard output log         |
| dashboard_8765.err.log           | Base dashboard error log                   |
| dashboard_8765_v5901.out.log     | v5.90.1 specific standard output           |
| dashboard_8765_v5901.err.log     | v5.90.1 specific error output              |
| dashboard_8765_v5902.out.log     | v5.90.2 specific standard output (current) |
| dashboard_8765_v5902.err.log     | v5.90.2 specific error output (current)    |
| dashboard_8765_hardening.out.log | System hardening pass standard output      |

| Log File                         | Purpose                            |
|----------------------------------|------------------------------------|
| dashboard_8765_hardening.err.log | System hardening pass error output |

**Note**

Port 8765 is the default dashboard port. All log files follow the naming convention `dashboard_{port}[_{version_or_phase}].{out|err}.log`.

# Section 2: Module Reference

This section documents each module in the Claire platform. For every module, the following information is provided: module path, file inventory (filename and purpose), architectural role, stage mapping within the 30-stage lifecycle, and key inter-module dependencies.

## 2.1 ingestion/ — Signal Ingestion & Preprocessing

**Path:** `src/claire/ingestion/`

| File                       | Purpose                                                                              |
|----------------------------|--------------------------------------------------------------------------------------|
| <code>loaders.py</code>    | Signal source loading — reads from various input formats and source types            |
| <code>preprocess.py</code> | Signal preprocessing and cleaning — normalization, deduplication, noise reduction    |
| <code>validators.py</code> | Signal validation rules — schema compliance, completeness checks, format enforcement |
| <code>__init__.py</code>   | Module initialization and public interface exports                                   |

**Architectural Role:** First contact point for all incoming signals. This module handles loading from various source formats, preprocessing/cleaning, and initial validation before signals enter the lifecycle pipeline.

**Stage Mapping:** Stage 1 (Signal Ingestion), Stage 2 (Signal Normalization)

**Key Dependencies:** connectors/ (for source-specific loading), models/ (for signal data models)

## 2.2 connectors/ — Data Source Connectors

**Path:** src/claire/connectors/

| File                 | Purpose                                                                                  |
|----------------------|------------------------------------------------------------------------------------------|
| base.py              | Base connector interface — abstract class defining the connector contract                |
| connector_hub.py     | Connector orchestration hub — coordinates multiple connectors for parallel data fetching |
| connector_manager.py | Connector lifecycle management — initialization, health checks, graceful shutdown        |
| academic.py          | Academic/research data source connector — scholarly articles, preprints, citations       |
| financial.py         | Financial market data connector — price feeds, SEC filings, financial statements         |
| market.py            | Market intelligence data connector — industry reports, market sizing, competitive data   |
| news.py              | News and media source connector — real-time news feeds, press releases, media monitoring |
| patent.py            | Patent database connector — patent filings, claims, prior art searches                   |
| web_fetcher.py       | General web data fetching — HTTP client for arbitrary web sources                        |
| manager.py           | Connector pool management — connection pooling, rate limit                               |

| File                     | Purpose                                            |
|--------------------------|----------------------------------------------------|
|                          | coordination                                       |
| <code>__init__.py</code> | Module initialization and public interface exports |

**Architectural Role:** Abstraction layer for all external data sources. Each connector type handles source-specific authentication, data fetching, rate limiting, and format normalization. The connector hub orchestrates multiple connectors, and the connector manager handles lifecycle operations (initialization, health checks, shutdown).

**Stage Mapping:** Stage 3 (Source Validation & Weighting)

**Key Dependencies:** `config/` (for API keys and credentials), `ingestion/` (feeds data into the ingestion pipeline)

|                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Dual-Version Confirmation</b></p> <p>Both Python 3.11 and 3.13 bytecode files exist for <code>connector_manager</code>, <code>financial</code>, <code>market</code>, and <code>patent</code> — confirming dual-version testing coverage for critical connector code paths.</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 2.3 `core/` — Core Processing Engine

**Path:** `src/claire/core/`

| File                        | Purpose                                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------------------|
| <code>data_engine.py</code> | Central data processing — routing, transformation, and consolidation of processed signals     |
| <code>gateway.py</code>     | System gateway/entry point — dispatches incoming data to the appropriate processing path      |
| <code>interpreter.py</code> | Signal interpretation and thesis formation — converts analyzed signals into structured theses |



| File                             | Purpose                                                                                                   |
|----------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>orreadir.py</code>         | Orchestration/direction logic — controls processing flow and decision routing                             |
| <code>planner.py</code>          | Execution planning and routing — determines optimal paths through the lifecycle                           |
| <code>semantic.py</code>         | Semantic analysis and meaning extraction — NLP-driven understanding of signal content                     |
| <code>signal_amplifier.py</code> | Signal strength amplification and context enrichment — strengthens weak signals through cross-referencing |
| <code>__init__.py</code>         | Module initialization and public interface exports                                                        |

**Architectural Role:** The brain of the system. This module handles signal amplification (strengthening weak signals through context enrichment), semantic analysis (extracting meaning from raw data), interpretation (forming theses from analyzed signals), and planning (determining execution routes through the lifecycle).

**Stage Mapping:** Stage 4 (Context Expansion), Stage 5 (Signal Consolidation), Stage 10 (Insight/Thesis Structuring)

**Key Dependencies:** `ingestion/` (receives processed signals), `live_intelligence/` (feeds into real-time analysis), `lifecycle/` (governed by stage contracts)

## 2.4 `live_intelligence/` — Real-Time Intelligence Engine

**Path:** `src/claire/live_intelligence/`

| File                            | Purpose                                                                    |
|---------------------------------|----------------------------------------------------------------------------|
| <code>connectors.py</code>      | Live data connections — real-time stream management and connection pooling |
| <code>entity_registry.py</code> | Entity tracking and deduplication — maintains a canonical registry of all  |

| File                        | Purpose                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------|
|                             | identified entities                                                                                          |
| gap_detection.py            | Market gap identification — detects unserved or underserved market needs from signal patterns                |
| history_store.py            | Historical data and run history — persists past run data for trend comparison and backtesting                |
| live_opportunity_monitor.py | Real-time opportunity tracking — continuous monitoring for emerging opportunities                            |
| monitor_candidate_bridge.py | Bridge between monitoring and candidate identification — connects detected patterns to actionable candidates |
| signal_extraction.py        | Live signal extraction from streams — real-time parsing and signal isolation                                 |
| solution_synthesis.py       | Solution generation from gaps and opportunities — auto-generates candidate solutions                         |
| source_scan_planner.py      | Planning which sources to scan — optimizes source coverage based on domain and signal type                   |
| trend_clustering.py         | Trend identification and grouping — clusters related signals into coherent trend narratives                  |
| __init__.py                 | Module initialization and public interface exports                                                           |

**Architectural Role:** The largest and most complex module in the Claire platform. Handles real-time signal extraction, entity tracking, trend clustering, gap detection, opportunity monitoring, solution synthesis, and the bridge between monitoring and candidate identification. This is where raw intelligence becomes actionable output.

**Stage Mapping:** Stage 6 (Entity Extraction), Stage 7 (Relationship Mapping), Stage 8 (Trend Discovery), Stage 9 (Cluster Formation), Stage 11 (Gap Detection), Stage 12 (Gap Qualification), Stages 15–16 (Advancement/Solution), Stage 23 (Market Positioning)

**Key Dependencies:** `core/` (receives interpreted signals), `concepts/` (feeds into for breakthrough filtering), `acquirers/` (feeds candidates to), `lifecycle/` (governed by stage contracts)

### Bytecache Coverage

All 10 source files have Python 3.13 bytecache present, confirming all files are actively compiled and tested under the latest supported runtime.

## 2.5 concepts/ — Concept Generation & Breakthrough Filtering

**Path:** `src/claire/concepts/`

| File                                | Purpose                                                                                             |
|-------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>breakthrough_filter.py</code> | Breakthrough identification and classification — evaluates candidates against 60+ category taxonomy |
| <code>constraints.py</code>         | Constraint definition and enforcement — ensures generated concepts are grounded and feasible        |
| <code>generator.py</code>           | Concept and discovery generation — produces candidate concepts from gap analysis data               |

**Architectural Role:** Handles the critical transition from gap analysis to discovery. The generator produces candidate concepts and discoveries. The breakthrough filter evaluates them against the 60+ category taxonomy. Constraints ensure generated concepts are grounded and feasible within real-world parameters.

**Stage Mapping:** Stage 13 (Discovery Generation), Stage 14 (Breakthrough Identification & Classification)

**Key Dependencies:** `live_intelligence/` (receives gaps and opportunities), `models/breakthrough.py` (breakthrough data model), `lifecycle/` (governed by stage contracts)

## 2.6 design/ — Design Portal

**Path:** `src/claire/design/`

| File                   | Purpose                                                                                      |
|------------------------|----------------------------------------------------------------------------------------------|
| <code>portal.py</code> | Design portal output generation — blueprints, specifications, and technical design documents |

**Architectural Role:** Produces the design and blueprint output for validated breakthroughs. Takes feasibility-validated solutions and generates structured design documents, specifications, and implementation blueprints.

**Stage Mapping:** Stage 22 (Design Portal Output / Blueprints / Specs)

**Key Dependencies:** `concepts/` (receives validated concepts), `deals/` (feeds into for deal packaging)

## 2.7 acquirers/ — Acquirer Intelligence

**Path:** `src/claire/acquirers/`

| File                     | Purpose                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------|
| <code>dataset.py</code>  | Acquirer database and profiles — maintains the registry of potential acquirer organizations |
| <code>matcher.py</code>  | Acquirer-target matching logic — evaluates alignment between outputs and acquirer interests |
| <code>scoring.py</code>  | Acquisition fit scoring — produces ranked acquirer lists with quantified fit rationale      |
| <code>__init__.py</code> | Module initialization and public interface exports                                          |

**Architectural Role:** Identifies potential acquirers for breakthrough outputs and scores their fit. The dataset maintains acquirer profiles, the matcher evaluates

alignment between outputs and acquirer interests, and scoring produces ranked acquirer lists with rationale.

**Stage Mapping:** Stage 28 (Acquirer Identification), Stage 29 (Acquisition Fit & Rationale)

**Key Dependencies:** `live_intelligence/` (receives market intelligence), `deals/` (coordinates with for package construction), `lifecycle/` (governed by stage contracts)

## 2.8 deals/ — Deal Construction

**Path:** `src/claire/deals/`

| File                        | Purpose                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------|
| <code>deal_scorer.py</code> | Deal quality and viability scoring — quantitative assessment of deal structures                |
| <code>discovery.py</code>   | Deal opportunity discovery — identifies optimal deal structures from available data            |
| <code>rationale.py</code>   | Acquisition rationale generation — produces narrative and quantitative justification documents |

**Architectural Role:** Constructs the deal layer of acquisition packages. Scores deal viability, discovers deal structures, and generates acquisition rationale documents. Works in coordination with `acquirers/` to produce complete acquisition-ready packages.

**Stage Mapping:** Stage 29 (Acquisition Fit & Rationale), Stage 30 (Final Package Construction)

**Key Dependencies:** `acquirers/` (receives acquirer matches), `design/` (receives design specs), `lifecycle/` (governed by stage contracts)

## 2.9 analysis/ — Pattern Analysis & Feature Extraction

**Path:** src/claire/analysis/

| File                 | Purpose                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------|
| demand_model.py      | Market demand modeling — quantitative assessment of market demand for identified opportunities  |
| feature_extractor.py | Signal feature extraction — identifies key characteristics, patterns, and attributes of signals |
| pattern_engine.py    | Pattern detection and analysis — detects recurring patterns across signal streams               |
| __init__.py          | Module initialization and public interface exports                                              |

### Sub-module: analysis/bridge/

| File          | Purpose                                                                            |
|---------------|------------------------------------------------------------------------------------|
| masterpass.py | Bridge between analysis layers — connects analysis outputs to downstream consumers |
| __init__.py   | Sub-module initialization                                                          |

**Architectural Role:** Provides analytical depth to the pipeline. The pattern engine detects recurring patterns across signals. The feature extractor identifies key characteristics of signals and trends. The demand model assesses market demand for identified opportunities. The bridge sub-module connects analysis outputs to other system layers.

**Stage Mapping:** Supports Stages 8–9 (Trend Discovery / Clustering), Stages 17–21 (Solution Assessment)

**Key Dependencies:** core/ (receives processed signals), live\_intelligence/ (feeds pattern data to)

## 2.10 lifecycle/ — Governance Spine

### CRITICAL MODULE

The lifecycle module is the governance spine of the entire Claire platform. Every signal flows through `lifecycle_runner`. Every stage must satisfy its contract. Every transition must pass the completion gate. Every decision is recorded for audit. Changes to this module require the highest level of review.

**Path:** `src/claire/lifecycle/`

| File                                 | Purpose                                                                                         |
|--------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>lifecycle_runner.py</code>     | Executes the 30-stage pipeline sequentially — the main execution loop                           |
| <code>stage_contracts.py</code>      | Defines what each stage must produce — input/output contracts with type and quality constraints |
| <code>contract_validator.py</code>   | Enforces contract compliance at each gate — validates outputs against stage contracts           |
| <code>completion_gate.py</code>      | Decides if a stage passes or blocks — binary gate enforcement at each stage boundary            |
| <code>stage_registry.py</code>       | Maps stage names to implementations — registry pattern for stage discovery and loading          |
| <code>lifecycle_registry.py</code>   | Tracks full lifecycle state across runs — global view of all lifecycle executions               |
| <code>lifecycle_context.py</code>    | Carries state between stages during a run — the context object passed through the pipeline      |
| <code>stage_status.py</code>         | Per-stage status tracking — pass/fail/skip status with timing and metadata                      |
| <code>threshold_provenance.py</code> | Records WHY decisions were made — full traceability of every gate decision                      |
| <code>__init__.py</code>             | Module initialization and public                                                                |

| File | Purpose           |
|------|-------------------|
|      | interface exports |

**Architectural Role:** The most critical module in the system. This is the governance spine that enforces the entire 30-stage lifecycle. Every signal flows through `lifecycle_runner`. Every stage must satisfy its contract (validated by `contract_validator` against `stage_contracts`). Every transition must pass the `completion_gate`. Every decision is recorded by `threshold_provenance` for audit. The `lifecycle_context` carries state between stages, and `lifecycle_registry` maintains the global view of all lifecycle runs.

**Stage Mapping:** Governs ALL 30 stages

**Key Dependencies:** Every other module depends on `lifecycle/` for governance enforcement.

### Testing Coverage

All 9 source files have Python 3.13 bytecache. 8 of 9 also have Python 3.11 bytecache. This is the most thoroughly tested module in the system, reflecting its critical governance role.

## 2.11 api/ — HTTP API Layer

**Path:** `src/claire/api/`

| File                      | Purpose                                                                                |
|---------------------------|----------------------------------------------------------------------------------------|
| <code>server.py</code>    | HTTP server initialization and configuration — aiohttp application setup               |
| <code>router.py</code>    | Central route registration — aggregates all route modules into a unified routing table |
| <code>endpoints.py</code> | Endpoint definitions — shared endpoint utilities and base handlers                     |



| File                              | Purpose                                                                          |
|-----------------------------------|----------------------------------------------------------------------------------|
| <code>schemas.py</code>           | Request/response Pydantic schemas — API contract enforcement via Pydantic 2.13.3 |
| <code>commands.py</code>          | Command processing — parses and dispatches API commands to system operations     |
| <code>routes_pipeline.py</code>   | Pipeline execution endpoints — trigger runs, query status, retrieve results      |
| <code>routes_acquirers.py</code>  | Acquirer query endpoints — search, match, and score acquirers                    |
| <code>routes_dashboard.py</code>  | Dashboard data endpoints — feeds data to the browser-based dashboard             |
| <code>routes_history.py</code>    | Run history endpoints — browse and compare historical pipeline runs              |
| <code>routes_connectors.py</code> | Data connector management endpoints — enable, disable, and monitor connectors    |
| <code>routes_platform.py</code>   | Platform status endpoints — version, health, and system information              |
| <code>routes_system.py</code>     | System health and configuration endpoints — runtime configuration management     |
| <code>routes_proxy.py</code>      | Proxy/forwarding endpoints — request forwarding for federated deployments        |
| <code>routes_command.py</code>    | Command execution endpoints — execute system commands via API                    |
| <code>routes_update.py</code>     | System update endpoints — check for and apply system updates                     |
| <code>__init__.py</code>          | Module initialization and public interface exports                               |

**Architectural Role:** The external interface to Claire. Provides HTTP APIs for all system operations: triggering pipeline runs, querying acquirers, viewing dashboard data, browsing run history, managing data connectors, checking platform status, and executing commands. Uses aiohttp for fully async operation.

**Stage Mapping:** Entry point to all stages (via `routes_pipeline.py`)

**Key Dependencies:** All modules — routes expose functionality from every module in the system.

## 2.12 learning/ — Recursive Self-Ingestion Foundation

**Path:** `src/claire/learning/`

| File                       | Purpose                                                                               |
|----------------------------|---------------------------------------------------------------------------------------|
| <code>feedback.py</code>   | Feedback loop processing — analyzes completed run outcomes for improvement signals    |
| <code>metrics.py</code>    | Performance metric tracking — accumulates quality and performance metrics across runs |
| <code>retraining.py</code> | Model/logic retraining — adjusts system behavior and thresholds from accumulated data |

**Architectural Role:** The foundation for Claire's recursive self-ingestion moat. Feedback processes completed run outcomes. Metrics tracks performance across runs. Retraining adjusts system behavior based on accumulated learning. This module is currently foundational — full recursive self-ingestion is planned for v5.98-5.99.

**Stage Mapping:** Post-Stage 30 (feeds completed outputs back to Stage 1)

**Key Dependencies:** `lifecycle/` (receives completion data), `ingestion/` (feeds back into the pipeline)

## 2.13 mode/ — Runtime Mode Management

**Path:** `src/claire/mode/`

| File                            | Purpose                                                                            |
|---------------------------------|------------------------------------------------------------------------------------|
| <code>isolation_layer.py</code> | Runtime isolation between modes — ensures modes do not interfere with shared state |

| File                              | Purpose                                                                                        |
|-----------------------------------|------------------------------------------------------------------------------------------------|
| <code>state_manager.py</code>     | System state management — tracks and persists system state across mode transitions             |
| <code>switch_controller.py</code> | Mode switching logic — handles transitions between Standard, Desktop, Live, and Portable modes |

**Architectural Role:** Manages the four runtime modes (Standard, Desktop, Live, Portable). The isolation layer ensures modes don't interfere with each other. The state manager tracks system state. The switch controller handles transitions between modes.

**Stage Mapping:** System-level (not stage-specific)

#### Dual-Version Confirmation

All 3 files have both Python 3.11 and 3.13 bytecode — tested in both supported Python versions.

## 2.14 config/ — Configuration Management

**Path:** `src/claire/config/`

| File                       | Purpose                                                                           |
|----------------------------|-----------------------------------------------------------------------------------|
| <code>logging.py</code>    | Logging configuration — log levels, formatters, handlers, rotation policies       |
| <code>security.yaml</code> | Security settings and policies — authentication, authorization, rate limits, CORS |
| <code>__init__.py</code>   | Module initialization and configuration loading                                   |

**Sub-directory:** `config/env/`

| File                  | Purpose                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------|
| <code>dev.env</code>  | Development environment variables — local sources, verbose logging, debug mode            |
| <code>prod.env</code> | Production environment variables — production API keys, structured logging, strict limits |

**Architectural Role:** Centralized configuration management. Logging configuration, security policies, and environment-specific settings are all managed here. Separation of `dev` and `prod` environments ensures safe development without affecting production.

**Stage Mapping:** System-level (not stage-specific)

## 2.15 infra/ — Deployment Infrastructure

**Path:** `src/claire/infra/`

**Sub-directory:** `infra/docker/`

| File                            | Purpose                                                                                     |
|---------------------------------|---------------------------------------------------------------------------------------------|
| <code>Dockerfile</code>         | Container image definition — Python base image, dependency install, source copy, entrypoint |
| <code>docker-compose.yml</code> | Multi-container orchestration — Claire app + nginx proxy, volumes, networks, env injection  |

**Sub-directory:** `infra/nginx/`

| File                    | Purpose                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------|
| <code>nginx.conf</code> | Reverse proxy configuration — SSL termination, load balancing, static assets, upstream health |

**Architectural Role:** Production deployment infrastructure. Docker containerization enables consistent deployment across environments. nginx serves as a reverse proxy for the aiohttp API server, handling SSL termination, load balancing, and static asset serving.

**Stage Mapping:** System-level (deployment)

## 2.16 models/ — Data Models

**Path:** `src/claire/models/`

| File                         | Purpose                                                                                                                                   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>breakthrough.py</code> | Breakthrough data model — defines the 60+ category taxonomy structure, classification schema, and scoring fields                          |
| <i>Additional files</i>      | Additional Pydantic models for signals, entities, trends, deals, and other pipeline data structures (tree truncated in codebase analysis) |

**Architectural Role:** Pydantic data models that define the shape of data flowing through the pipeline. The breakthrough model is particularly important as it defines the 60+ category taxonomy structure. All models use Pydantic 2.13.3 for runtime validation and serialization.

**Stage Mapping:** Used by all stages (data contracts)

## 2.17 bridge/ — Cross-Module Orchestration

**Path:** `src/claire/bridge/`

| File                       | Purpose                                                                                           |
|----------------------------|---------------------------------------------------------------------------------------------------|
| <code>masterpass.py</code> | Master orchestration pass — coordinates data movement between modules without direct dependencies |
| <code>__init__.py</code>   | Module initialization and public interface exports                                                |

**Architectural Role:** Handles cross-module data flow and orchestration. The masterpass coordinates data movement between modules that don't have direct dependencies but need to share information during pipeline execution.

**Stage Mapping:** System-level (cross-cutting)

---

## Section 3: Stage-to-Module Mapping

The following table maps every stage in Claire's 30-stage lifecycle to the primary module responsible for execution and the key files involved. This mapping is the canonical reference for determining which code paths are exercised at each stage.

| Stage | Name                          | Primary Module                  | Key Files                                       |
|-------|-------------------------------|---------------------------------|-------------------------------------------------|
| 1     | Signal Ingestion              | ingestion/                      | loaders.py, preprocess.py                       |
| 2     | Signal Normalization          | ingestion/                      | preprocess.py, validators.py                    |
| 3     | Source Validation & Weighting | connectors/                     | connector_hub.py, connector_manager.py, base.py |
| 4     | Context Expansion             | core/                           | signal_amplifier.py, semantic.py                |
| 5     | Signal Consolidation          | core/                           | data_engine.py, gateway.py                      |
| 6     | Entity Extraction             | live_intelligence /             | entity_registry.py, signal_extraction.py        |
| 7     | Relationship Mapping          | live_intelligence /             | monitor_candidate_bridge.py, history_store.py   |
| 8     | Trend Discovery               | live_intelligence / + analysis/ | trend_clustering.py, pattern_engine.py          |
| 9     | Cluster Formation             | live_intelligence / + analysis/ | trend_clustering.py,                            |

| Stage | Name                           | Primary Module                  | Key Files                                 |
|-------|--------------------------------|---------------------------------|-------------------------------------------|
|       |                                |                                 | feature_extractor.py                      |
| 10    | Insight / Thesis Structuring   | core/                           | interpreter.py, planner.py, semantic.py   |
| 11    | Gap Detection                  | live_intelligence /             | gap_detection.py                          |
| 12    | Gap Qualification              | live_intelligence /             | gap_detection.py, source_scan_planner.py  |
| 13    | Discovery Generation           | concepts/                       | generator.py, constraints.py              |
| 14    | Breakthrough Identification    | concepts/ + models/             | breakthrough_filter.py, breakthrough.py   |
| 15    | Advancement Path Selection     | core/ + lifecycle/              | planner.py, stage_registry.py             |
| 16    | Auto Invention                 | live_intelligence /             | solution_synthesis.py                     |
| 17    | Solution Structuring           | analysis/ + deals/              | demand_model.py, deal_scorer.py           |
| 18    | Buildability Assessment        | analysis/                       | demand_model.py                           |
| 19    | Viability Assessment           | analysis/ + deals/              | demand_model.py, deal_scorer.py           |
| 20    | Manufacturability Assessment   | analysis/                       | demand_model.py                           |
| 21    | Feasibility Validation         | lifecycle/                      | contract_validator.py, completion_gate.py |
| 22    | Design Portal Output           | design/                         | portal.py                                 |
| 23    | Market Positioning             | live_intelligence /             | live_opportunity_monitor.py               |
| 24    | Moat & Differentiation         | live_intelligence / + analysis/ | source_scan_planner.py                    |
| 25    | Business Model & Value Capture | deals/ + analysis/              | rationale.py, demand_model.py             |

| Stage | Name                        | Primary Module              | Key Files                                         |
|-------|-----------------------------|-----------------------------|---------------------------------------------------|
| 26    | Competitor Analysis         | live_intelligence /         | source_scan_planner.py                            |
| 27    | Portfolio Creation          | live_intelligence / + core/ | live_opportunity_monitor.py, planner.py           |
| 28    | Acquirer Identification     | acquirers/                  | dataset.py, matcher.py                            |
| 29    | Acquisition Fit & Rationale | acquirers/ + deals/         | scoring.py, rationale.py                          |
| 30    | Final Package Construction  | deals/ + lifecycle/         | deal_scorer.py, discovery.py, lifecycle_runner.py |

## Section 4: API Route Reference

The Claire API is organized into domain-specific route modules, each registered through the central `router.py`. All endpoints are served by `aiohttp` and validated via Pydantic schemas defined in `schemas.py`.

| Route Module                      | Purpose                           | Likely Endpoints                                                                                          |
|-----------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>routes_pipeline.py</code>   | Pipeline execution and management | POST <code>/pipeline/run</code> , GET <code>/pipeline/status</code> , GET <code>/pipeline/results</code>  |
| <code>routes_acquirers.py</code>  | Acquirer queries and matching     | GET <code>/acquirers</code> , GET <code>/acquirers/match</code> , GET <code>/acquirers/score</code>       |
| <code>routes_dashboard.py</code>  | Dashboard data feeds              | GET <code>/dashboard</code> , GET <code>/dashboard/metrics</code> , GET <code>/dashboard/runs</code>      |
| <code>routes_history.py</code>    | Run history browsing              | GET <code>/history</code> , GET <code>/history/{run_id}</code> , GET <code>/history/compare</code>        |
| <code>routes_connectors.py</code> | Data connector management         | GET <code>/connectors</code> , POST <code>/connectors/enable</code> , GET <code>/connectors/status</code> |



| Route Module                    | Purpose                  | Likely Endpoints                                                                                                 |
|---------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>routes_platform.py</code> | Platform status and info | GET <code>/platform/status</code> ,<br>GET <code>/platform/version</code> ,<br>GET <code>/platform/health</code> |
| <code>routes_system.py</code>   | System configuration     | GET <code>/system/config</code> , POST <code>/system/settings</code>                                             |
| <code>routes_proxy.py</code>    | Request forwarding/proxy | POST <code>/proxy/forward</code>                                                                                 |
| <code>routes_command.py</code>  | Command execution        | POST <code>/command/execute</code>                                                                               |
| <code>routes_update.py</code>   | System updates           | POST <code>/update/check</code> , POST <code>/update/apply</code>                                                |

### Schema Enforcement

All request and response payloads are validated against Pydantic schemas defined in `schemas.py`. Invalid requests receive a 422 `Unprocessable Entity` response with field-level error details. Schema definitions serve as the API contract for external consumers.

# Section 5: Governance Spine Reference

## 5.1 Lifecycle Runner Execution Flow

The `lifecycle_runner.py` orchestrates the complete 30-stage pipeline. The execution follows a strict sequential flow with contract enforcement at every stage boundary.

### Execution Sequence:

- Initialize `lifecycle_context`** — Create a fresh context object to carry state through the pipeline. Assign a unique run ID and timestamp.
- For each stage (1–30), execute the following loop:**

1. **Load stage** from stage\_registry — Resolve the stage name to its implementation module and function.
  2. **Validate input contract** — contract\_validator checks the current lifecycle\_context against the stage's input requirements defined in stage\_contracts.
  3. **Execute stage logic** — Invoke the stage implementation with the current context.
  4. **Validate output contract** — contract\_validator checks the stage's output against the required output fields, types, and quality thresholds.
  5. **Check completion\_gate** — Binary pass/fail decision on whether the stage output meets advancement criteria.
  6. **Record decision provenance** — threshold\_provenance logs the full decision trail: evidence, thresholds, alternatives considered.
  7. **Update stage\_status** — Record pass/fail/skip with timing metadata.
  8. **Pass context to next stage** — Update lifecycle\_context with stage outputs and advance.
3. **On completion:** Update lifecycle\_registry with the final run state. Trigger the learning/ feedback loop to process run outcomes.

### **Gate Failure Behavior**

If a completion\_gate fails, the pipeline blocks at that stage. The failure is recorded with full provenance (what was expected, what was received, why it failed). The run may be retried from the failed stage after remediation.

## 5.2 Stage Contract Structure

Each stage contract is defined in `stage_contracts.py` and specifies the complete input/output requirements for that stage. Contracts are the enforceable specification of the pipeline's data flow.

| Contract Element               | Description                                                                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Required Inputs</b>         | Data fields and types that must be present in the <code>lifecycle_context</code> before stage execution. Missing inputs trigger an immediate gate failure. |
| <b>Required Outputs</b>        | Data fields and types that the stage must produce and write to the <code>lifecycle_context</code> . Missing outputs trigger a post-execution gate failure. |
| <b>Quality Thresholds</b>      | Minimum scores, confidence levels, or completeness metrics that outputs must meet. Thresholds are numeric and machine-enforceable.                         |
| <b>Governance Gates</b>        | Additional conditions that must pass — cross-field validations, consistency checks, or domain-specific rules.                                              |
| <b>Provenance Requirements</b> | What must be recorded for audit: decision rationale, evidence sources, threshold comparisons, alternative paths considered.                                |

## 5.3 Threshold Provenance

The `threshold_provenance.py` module records a complete decision trail for every gate decision in the lifecycle. This ensures full traceability and supports post-run audit and analysis.

**For every decision, the following is recorded:**

| Provenance Field | Description                                                 |
|------------------|-------------------------------------------------------------|
| <b>Decision</b>  | What was decided — pass, fail, skip, or conditional advance |
| <b>Evidence</b>  | What evidence supported the decision                        |

| Provenance Field        | Description                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------|
|                         | — data values, scores, signal counts                                                      |
| Threshold Applied       | What threshold was applied — the numeric or categorical bar that was set                  |
| Threshold Met           | Whether the threshold was met — boolean with the actual vs. required values               |
| Alternatives Considered | What alternative paths were considered — branching options and why they were not selected |
| Timestamp & Context     | When the decision was made and the full context state at decision time                    |

---

# Section 6: Configuration Reference

## 6.1 Environment Variables

Environment variables are defined in `.env` (runtime) and `.env.example` (template). The following table documents the expected variable categories and their roles.

| Variable Category  | Example Key                    | Purpose                                                 |
|--------------------|--------------------------------|---------------------------------------------------------|
| Financial Data API | <code>FINANCIAL_API_KEY</code> | Authentication key for financial market data connector  |
| Market Data API    | <code>MARKET_API_KEY</code>    | Authentication key for market intelligence connector    |
| News API           | <code>NEWS_API_KEY</code>      | Authentication key for news and media connector         |
| Academic API       | <code>ACADEMIC_API_KEY</code>  | Authentication key for academic/research data connector |

| Variable Category    | Example Key      | Purpose                                                           |
|----------------------|------------------|-------------------------------------------------------------------|
| <b>Patent API</b>    | PATENT_API_KEY   | Authentication key for patent database connector                  |
| <b>Database</b>      | DATABASE_URL     | Database connection string for persistence layer                  |
| <b>Server</b>        | HOST, PORT       | Server bind address and port (default: 0.0.0.0:8765)              |
| <b>Logging</b>       | LOG_LEVEL        | Logging verbosity: DEBUG, INFO, WARNING, ERROR                    |
| <b>Feature Flags</b> | ENABLE_LEARNING  | Toggle for experimental features (e.g., recursive self-ingestion) |
| <b>Security</b>      | API_SECRET_TOKEN | Secret token for API authentication                               |

## 6.2 Security Configuration (security.yaml)

The `security.yaml` file defines security policies, access controls, and protection mechanisms for the Claire platform.

- **Authentication:** API token validation, session management, token expiration policies
- **Authorization:** Role-based access control for API endpoints and system commands
- **Rate Limiting:** Per-endpoint and per-client rate limit definitions to prevent abuse
- **CORS Configuration:** Allowed origins, methods, and headers for cross-origin requests
- **Input Sanitization:** Rules for sanitizing API inputs to prevent injection attacks
- **Audit Logging:** Security event logging — authentication attempts, authorization failures, rate limit hits

## 6.3 Development vs Production

| Setting      | Development (dev.env)                             | Production (prod.env)                                |
|--------------|---------------------------------------------------|------------------------------------------------------|
| Data Sources | Local/mock data sources                           | Production API keys with live data connections       |
| Logging      | Verbose (DEBUG level), console output             | Structured (INFO level), JSON format, file output    |
| Debug Mode   | Enabled — full stack traces, hot reload           | Disabled — minimal error exposure                    |
| Rate Limits  | Relaxed — higher limits for development iteration | Strict — production-grade protection                 |
| Performance  | Unoptimized — prioritizes debugging               | Optimized — connection pooling, caching, compression |
| Security     | Relaxed CORS, optional auth                       | Strict CORS, mandatory auth, TLS required            |

---

## Section 7: Deployment Reference

### 7.1 Docker Configuration

**Dockerfile** — Located at `src/claire/infra/docker/Dockerfile`

| Build Stage   | Description                                                                      |
|---------------|----------------------------------------------------------------------------------|
| Base Image    | Python 3.13 slim base image (with 3.11 compatibility verified)                   |
| Dependencies  | Install from <code>requirements.txt</code> — pinned versions for reproducibility |
| Source Copy   | Copy <code>src/claire/</code> into container at <code>/app/src/claire/</code>    |
| Configuration | Copy <code>.env</code> and configuration files, set environment variables        |
| Entry Point   | <code>python main.py</code> — starts the aiohttp                                 |

| Build Stage | Description |
|-------------|-------------|
|             | server      |

**docker-compose.yml** — Located at `src/claire/infra/docker/docker-compose.yml`

| Service              | Configuration                                                                                                    |
|----------------------|------------------------------------------------------------------------------------------------------------------|
| <b>claire</b> (app)  | Build from Dockerfile, expose port 8765, mount data volumes, inject environment variables from <code>.env</code> |
| <b>nginx</b> (proxy) | Use nginx image, mount <code>nginx.conf</code> , expose ports 80/443, depend on <code>claire</code> service      |

## 7.2 nginx Configuration

The `nginx.conf` file configures the nginx reverse proxy that sits in front of the aiohttp application server.

| Directive                | Purpose                                                         |
|--------------------------|-----------------------------------------------------------------|
| <b>Upstream</b>          | Define the aiohttp backend at <code>localhost:8765</code>       |
| <b>Reverse Proxy</b>     | Forward all <code>/api/</code> requests to the aiohttp upstream |
| <b>SSL Termination</b>   | Handle TLS at the proxy layer (production only)                 |
| <b>Static Assets</b>     | Serve dashboard HTML/CSS/JS files directly from nginx           |
| <b>Request Buffering</b> | Buffer large request bodies before proxying to the application  |
| <b>Health Checks</b>     | Upstream health check against <code>/platform/health</code>     |

## 7.3 CI/CD Pipeline

Continuous integration is managed via GitHub Actions, defined in `.github/workflows/ci.yml`.

| Pipeline Stage     | Actions                                                          |
|--------------------|------------------------------------------------------------------|
| Trigger            | On push to <code>main</code> and pull request branches           |
| Linting            | Code style enforcement and static analysis                       |
| Type Checking      | Pydantic model and type annotation verification                  |
| Test Execution     | Run pytest suite — unit tests, integration tests, contract tests |
| Build Verification | Docker image build and startup verification                      |
| Matrix             | Python 3.11 and 3.13 — both versions tested in CI                |

---

## Section 8: Output Package Reference

### 8.1 Standard Output Package (10 files)

Every completed pipeline run produces the following standard output package. All files are written to the run directory.

| Filename                    | Format   | Content Description                                                                                     |
|-----------------------------|----------|---------------------------------------------------------------------------------------------------------|
| <code>run_summary.md</code> | Markdown | Overview of the pipeline run — domain, signals processed, route taken, key findings, and timing metrics |



| Filename                    | Format   | Content Description                                                                                       |
|-----------------------------|----------|-----------------------------------------------------------------------------------------------------------|
| full_pipeline_output.json   | JSON     | Complete structured output from all 30 stages — the canonical machine-readable run record                 |
| portfolio_binder.md         | Markdown | Portfolio analysis, thesis, positioning, and recommendations                                              |
| opportunity_discovery.md    | Markdown | Discovered opportunities with evidence chains and scoring rationale                                       |
| strategic_positioning.md    | Markdown | Market positioning analysis and strategic recommendations                                                 |
| technical_feasibility.md    | Markdown | Technical feasibility assessment of identified opportunities — buildability, viability, manufacturability |
| deal_exit_model.md          | Markdown | Deal structure and exit model analysis — acquisition paths and valuation frameworks                       |
| productization_path.md      | Markdown | Path to productization for identified breakthroughs — milestones, dependencies, timeline                  |
| export_writer_manifest.json | JSON     | Manifest of all exported files and their metadata — serves as the run directory index                     |
| README.md                   | Markdown | Run documentation and file index — human-readable guide to the output package                             |

## 8.2 Enhanced Output Package (v5.90.2 additions — 8+ additional files)

Starting with v5.90.2, completed runs may produce an enhanced output package with the following additional files, depending on pipeline configuration and the data processed.

| Filename                         | Format   | Content Description                                                                                      |
|----------------------------------|----------|----------------------------------------------------------------------------------------------------------|
| acquirer_discovery_summary.md    | Markdown | Summary of identified potential acquirers with fit scores and match rationale                            |
| acquisition_readiness_summary.md | Markdown | Assessment of acquisition readiness — preparation needs, gaps, and timeline estimates                    |
| connected_context.json           | JSON     | Connected context data linking signals → trends → gaps → opportunities → outputs                         |
| diligence_checklist.json         | JSON     | Due diligence checklist for identified acquisition targets — structured checklist with completion status |
| governance_connected_summary.md  | Markdown | Governance gate results and decision provenance summary — human-readable audit trail                     |
| hybrid_fusion_summary.md         | Markdown | Hybrid analysis combining multiple intelligence streams into a unified narrative                         |
| normalized_signals.json          | JSON     | Normalized signal data with source weights, classifications, and confidence scores                       |
| source_manifest.json             | JSON     | Complete manifest of all sources used — classification, credibility scores, and contribution metrics     |

---

# Section 9: Run Directory Structure

Each pipeline run creates a timestamped directory using the following naming convention:

```
claire_run_{YYYYMMDD}_{HHMMSS}_{domain}_{subdomain}/
```

**Example:**

```
claire_run_20260502_001234_financial_market_intelligence_financial_signal_intelligence/
```

**Directory contents:** All output files from both the standard and enhanced output packages are placed in this directory. The `export_writer_manifest.json` serves as the authoritative index of all files in the run directory, including file paths, sizes, generation timestamps, and content hashes.

**Naming convention breakdown:**

| Segment     | Format         | Description                                                              |
|-------------|----------------|--------------------------------------------------------------------------|
| claire_run  | Literal prefix | Identifies the directory as a Claire pipeline run output                 |
| {YYYYMMDD}  | Date stamp     | Run date in ISO format (e.g., 20260502)                                  |
| {HHMMSS}    | Time stamp     | Run start time in 24-hour format (e.g., 001234)                          |
| {domain}    | Snake case     | Primary domain of the pipeline run (e.g., financial_market_intelligence) |
| {subdomain} | Snake case     | Subdomain or specialization (e.g., financial_signal_intelligence)        |

---

# Section 10: Known Gaps & Future Modules

The following table documents architectural components that are designed but not yet fully implemented. These are tracked for future development and represent planned expansions of the Claire platform.

| Component                                   | Current State         | Details                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Recursive Self-Ingestion (full loop)</b> | Foundation built      | Architecture is designed. Foundation exists in the learning/ module ( <code>feedback.py</code> , <code>metrics.py</code> , <code>retraining.py</code> ). Full recursive self-ingestion — where completed run outputs feed back into Stage 1 as first-class signals — is planned for <b>v5.98-5.99</b> . |
| <b>Live Market Data Feeds</b>               | Architecture ready    | The connector architecture (see <code>connectors/</code> ) fully supports live data providers. No live market data provider is currently connected. Connector interface in <code>base.py</code> is ready for integration.                                                                               |
| <b>Web-Based Search Provider</b>            | Planned               | Planned for capability expansion. Will integrate with web search APIs to augment signal ingestion with real-time web intelligence. <code>web_fetcher.py</code> provides the HTTP foundation.                                                                                                            |
| <b>Multi-Tenant Support</b>                 | Architecture supports | The <code>mode/</code> module's isolation layer and the lifecycle context model support multi-tenancy architecturally. Tenant isolation, per-tenant configuration, and                                                                                                                                  |

| Component                             | Current State | Details                                                                                                                                                                                                                       |
|---------------------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                       |               | tenant-scoped data are not yet implemented.                                                                                                                                                                                   |
| <b>Full Design Portal Output</b>      | Partial       | design/portal.py exists and produces basic design output (Stage 22). Full blueprint generation — including detailed technical specifications, implementation blueprints, and bill-of-materials output — is pending expansion. |
| <b>System Ingestion / Replacement</b> | Defined       | Architecturally defined as a future capability where Claire can ingest descriptions of existing systems and produce replacement architectures. Not implemented in any module as of v5.90.2.                                   |

---

**CONFIDENTIAL** — Claire Technical Codebook & Engineering Reference — v5.90.2

Prepared by Craig Morris — May 3, 2026

This document is the property of the Claire engineering team. Unauthorized distribution is prohibited.

End of Document